

AutoStack

Project Status Report — April 2026 · University Mentor Review

What is AutoStack? A one-click deployment platform that provisions Kubernetes or AWS infrastructure for any containerised application — with transparent upfront cost estimates, built-in security isolation, and AI-assisted error recovery.

Overall Progress

Dimension	Honest Assessment	Status
Core platform (auth, DB, K8s deploy)	Functional end-to-end on local cluster	✓ Working
AWS deployments via Terraform	Flow exists; Terraform template accuracy unverified	~ Partial
Cost estimation	Formula-based with known AWS rates; not live API	~ Partial
AI error analysis	Frontend component built; backend endpoint missing	✗ Incomplete
Multi-tenant isolation	Frontend filter only — no server-side enforcement yet	✗ Critical gap
Encryption key persistence	Regenerated on restart in current docker-compose	✗ Critical bug
Production readiness	Not tested under concurrent load; no SSL; no backups	✗ Not ready

Feature Completion — Honest Scorecard

The table below rates each claimed feature against what is verifiably in the codebase. **Done** means tested and working end-to-end. **Partial** means the structure exists but relies on stubs, hardcoded values, or an unverified backend. **Claimed / Not verified** means documentation or UI exists but the underlying implementation could not be confirmed.

Feature Area	Claimed	Realistic	Key Gap / Evidence
Foundation — user accounts, dashboard, DB	4 / 4	4 / 4	PocketBase handles this; genuinely solid
Kubernetes — deploy, logs, rollback, scale	8 / 8	6 / 8	Core deploy + logs work; network ingress & volume features untested e2e
AWS deployments (ECS, RDS, S3, CloudFront)	9 / 9	5 / 9	UI + Terraform templates exist; plan→apply flow not verified on real account
Security (encryption, isolation, RBAC)	7 / 7	3 / 7	AES key lost on restart; no server-side row filter; cluster-admin over-scope
Cost estimation & tracking	8 / 8	3 / 8	Formula-based UI exists; no live pricing API; Cost Explorer integration absent
AI error analysis & auto-recovery	5 / 5	1 / 5	Frontend component built; backend /api/intelligence/* endpoints do not exist
Advanced cost features (Infracost, email)	9 / 9	2 / 9	Infracost integration unconfirmed in codebase; email via Resend not verified
Critical security fixes (this update)	5 / 5	0 / 5	Report claims fixes; not reflected in codebase reviewed — may be in-progress
Reliability (graceful shutdown, timeouts)	5 / 5	4 / 5	Mostly present; concurrent deployment guard exists but WebSocket auth missing

Realistic Completion Estimate

Category	Claimed done	Realistic done	Realistic %
Foundation	4	4	100%
Kubernetes deployments	8	6	75%
AWS deployments	9	5	55%

Category	Claimed done	Realistic done	Realistic %
Security	7	3	43%
Cost estimation	8	3	38%
AI Intelligence	5	1	20%
Advanced cost + new fixes	14	2	14%
Reliability	5	4	80%
Production essentials (SSL, backup, monitoring)	0	0	0%
Developer tools (CI/CD, environments)	0	0	0%
Enterprise features (teams, RBAC)	0	0	0%
TOTAL	60	28	~28%

The original report claims 60% completion. Honest assessment of what is verifiably functional end-to-end places the project at approximately 28%. The difference is not fabrication — the structural work, UI components, and backend routes largely exist — but a significant portion of claimed features rely on stubs, hardcoded fallbacks, or frontend-only implementations without a working backend.

Critical Issues — Must Fix Before Real Users

Priority	Issue	Impact	Fix Complexity
P0	Encryption key regenerated on restart	All saved AWS credentials become unreadable after container restart	Low — rotate key in mounted secret
P0	No server-side multi-tenant filter	Any logged-in user can read other users' deployment and ProjectBase collection rules	Low to medium — add ProjectBase filter
P0	WebSocket connections unauthenticated	Anyone who knows a rollout ID can stream and modify rollout	Medium — validate JWT on WS handshake
P1	cluster-admin grants * on * resources	A bug in K8s controller can destroy the entire cluster	Medium — scope to required namespaces only
P1	Terraform state on local disk; no crash recovery	Local apply container crash leaves state corrupted; no rollback	Medium — S3 backend + state locking
P1	AI intelligence backend endpoints do not exist	Analysis and RecoveryDashboard pages high-attempt /api/intelligence/* handlers	High — implement
P2	Hardcoded cost estimates do not respond to input	Input at 25% CPU shows same price as 2 vCPU; destroys trust	Low — destroy, replace hardcoded number

What Genuinely Works — Verified

Component	Evidence of real implementation
User auth + project organisation	PocketBase handles auth natively; collections and migrations confirmed
Kubernetes deployment (nginx, basic containers)	Demonstrated live on minikube; pods start, logs stream via WebSocket
Rollout history + rollback	RolloutsTable component, client-go integration, manifest diff UI all present
Blueprint system (K8s)	Blueprint CRUD complete; YAML manifest editor (Monaco) functional
Real-time log streaming	LogStream.svelte + WebSocket backend confirmed; pods logs appear in UI
AWS credential storage structure	AES-256 encryption code exists; credential manager service present in codebase
Kubernetes map view	SvelteFlow visualisation with live WS events; genuinely impressive UI feature
Cost estimation UI (static)	CostEstimator.svelte renders; shows range and breakdown; needs formula backend

What Needs Real Implementation (Not Just Structure)

Item	Current state	What real implementation looks like
Live cost estimation	Hardcoded \$25-\$45 per blueprint	Formula: (vCPU × \$0.04048 + memGB × \$0.004445) × 730 + ALB + NAT — response time
AI error analysis backend	Frontend calls /api/intelligence/analysis endpoint that sends Terraform/K8s logs to Claude API, returns structured JSON findings	Go handler that sends 404
Multi-tenant security	PocketBase getFullList returns all records, no tenant isolation	PocketBase filter client sites: listRule = 'owner = @request.auth.id'
Terraform apply → real AWS infra	HTTP handler exists; Terraform templates exist; AWS creds on dev machine	End-to-end test on AWS dev acct: verify ECS cluster, ALB, and RDS are actually deployed
Cost Explorer actual tracking	ActualCosts collection schema exists; no data yet	Background job routine calling AWS Cost Explorer API daily; cost stored per deployment
SSL / custom domains	Planned; not started	cert-manager integration or ACM automation via Terraform module
Automated backups	Not started	SQLite WAL snapshot to S3 on schedule; Terraform state backup policy

AI/LLM Integration — Practical Roadmap

The project has correctly identified AI-assisted error analysis as a differentiator. Below is a prioritised, buildable plan — each item has a defined input/output contract so it can be implemented in isolation.

Use case	Input	Output	Priority
Deployment failure diagnosis	Terraform error log + deployment ID	Structured JSON: error type, root cause, 3 fix steps	Step 1, highest value
Pre-deploy security review	Generated Terraform plan JSON	List of security warnings (open ports, public buckets, trust policy, RDS)	Step 2, trust builder
Cost explanation in plain language	Aggregated breakdown object + deployment ID	One paragraph: why this costs X, how to reduce it	Step 3, differentiator
Blueprint generation from description	Free-text user description of what they want to deploy	Complete K8s manifests or Terraform HCL + cost estimate	Step 4, wizard complexity
Anomaly detection narrative	Actual vs estimated cost delta + baseline	Plain English alert: 'Your RDS cost is 3× the estimate because...'	Step 5, niche use..'

Realistic Timeline to Production-Ready

Phase	Duration	Key deliverables	Outcome
Critical fixes	Week 1–2	Persist encryption key · PocketBase collection rules · Scope AWS IAM roles	No S3 leaks · Fixed import bug
Core functionality verification	Week 3–4	End-to-end AWS deploy on real account · Formula-based cost calculation	Deployment works · Accurate cost
AI features live	Week 5–6	Implement /api/intelligence/* handlers · Pre-deploy security checks	AI view · Diagnosis actually works
Production essentials	Month 2–3	SSL via cert-manager · SQLite backup to S3 · Health checks	Secure · Stable · First real users
Developer experience	Month 3–5	GitHub webhook auto-deploy · Environment management · CI/CD	Competitive with Railway / Render for developer workflow
Enterprise readiness	Month 5–8	Team management · Audit log · SSO · Cost allocation by project	Scalable to teams and organisations

Bottom Line for Mentor Review

AutoStack has a sound architecture and a working local Kubernetes deployment path. The codebase is approximately **28% functionally complete** against the 100-feature plan — not 60%. The gap is not fabrication; it is the difference between structure existing and features being verifiable end-to-end.

The three highest-value engineering priorities are: (1) server-side multi-tenant authorisation, which is a security prerequisite for any real user; (2) the AI error analysis backend, which is the most technically novel aspect of the

project and is currently unreachable; and (3) end-to-end verification of the AWS Terraform deployment flow on a real AWS account, which is the core product promise.

With focused effort on the critical fixes (two weeks of work), the project reaches a state that is safe to demonstrate to real users and technically defensible as a research contribution in the areas of infrastructure automation and LLM-assisted operational intelligence.

Report prepared April 12, 2026 · Based on codebase review of github.com/Raj-glitch-max/AutoStack-GO-Svelte